

The CitaDel System

Fase 4 – Entrega Final

Sistemes Operatius

Grup 40: Paula Morales Tamayo i Adrià Romo Soriano

01/02/26

Curs 2025-2026

Taula de continguts

1. Disseny	3
1.1. Diagrames de processos	3
1.1.1. Procés Maester (Pare)	3
1.1.2. Consell d'Emissaris (Fills - <i>Envoys</i>)	3
1.1.3. Esquema de comunicació (IPC)	4
1.2. Estructures de dades	6
1.2.1. Struct Maester (Gestor global)	6
1.2.2. Struct Mission (Protocol de <i>pipe</i>)	6
1.2.3. Struct EnvoyStatus (Estat compartit)	7
1.2.4. Struct Rutas (Gestor rutes)	7
1.2.5. Struct Stock (Gestor stock)	7
1.2.6. Struct Comprarltem (gestor temporal stock)	8
1.2.7. Struct Trama (gestor de trames)	8
1.2.8. Struct StockArgs (gestor temporal enviament stock)	8
1.2.9. Struct Allies (gestor d'aliances)	8
1.2.10. Struct PendingPledge (gestor temporal de <i>pledge</i>)	9
1.3. Recursos del sistema utilitzats	9
1.3.1. Gestió de processos i fils	9
1.3.2. Comunicació entre processos (IPC)	9
1.3.3. Sincronització i exclusió mútua	10
1.3.4. Xarxa i sockets	10
1.4. Implementacions Opcionals i Millores de Robustesa	11
1.4.1. Clean Shutdown (Tancament Net)	11
1.4.2. Gestió Avançada de Memòria (Valgrind Clean)	11
1.4.3. Actualització d'Estoc en Temps Real (Consistency)	11
2. Problemes Observats	12
2.1. Segmentation Fault al tancament	12
2.2. Bloqueig d'envoys per manca de resposta	12
2.3. Leaks de memòria i dades no inicialitzades	13
2.4. Falsos Timeouts instantanis	13
3. Estimació temporal	14
3.1. Investigació	14
3.1.1. Fase 1 (5h)	14
3.1.2. Fase 2 (6h)	14
3.1.3. Fase 3 (10h)	14
3.1.4. Fase 4 (8h)	14
3.2. Disseny	14
3.2.1. Fase 1 (4h)	14
3.2.2. Fase 2 (5h)	15
3.2.3. Fase 3 (7h)	15
3.2.4. Fase 4 (5h)	15
3.3. Implementació	15
3.3.1. Fase 1 (12h)	15
3.3.2. Fase 2 (15h)	15
3.3.3. Fase 3 (22h)	15
3.3.4. Fase 4 (20h)	15

3.4.	Testing.....	15
3.4.1.	Fase 1 (3h)	15
3.4.2.	Fase 2 (6h)	16
3.4.3.	Fase 3 (10h).....	16
3.4.4.	Fase 4 (15h).....	16
3.5.	Documentació	16
	Fase 1 (2h)	16
	Fase 2 (3h).....	16
	Fase 3 (6h)	16
	Fase 4 (6h)	16
4.	<i>Conclusions i propostes de millora</i>	17
4.1.	Conclusions	17
4.1.1.	Canvi de Paradigma Arquitectònic.....	17
4.1.2.	Domini de la Comunicació Inter-Procés (IPC)	17
4.1.3.	Robustesa i Estabilitat	17
4.1.4.	Qualitat del Codi	17
4.2.	Propostes de millora	18
4.2.1.	Encaminament Dinàmic.....	18
4.2.2.	Seguretat i Encriptació.....	18
4.2.3.	Recàrrega de Configuració en Calent	18
4.2.4.	Interfície Gràfica de Monitorització	18
5.	<i>Tema de la pràctica</i>	19
6.	<i>Bibliografia</i>.....	21

1. Disseny

En aquesta fase final, el disseny del sistema ha evolucionat d'un model seqüencial (on el Maester feia totes les tasques una per una) a un model concurrent i distribuït. L'arquitectura s'ha reestructurat per delegar les tasques de comunicació sortint (Client) a processos fills anomenats *Envoys*, mentre que el Maester es concentra en la gestió, l'encaminament (Router) i la recepció de connexions (Servidor).

1.1. Diagrames de processos

El sistema s'estructura seguint un model Pare-Fill amb Delegació de Tasques.

1.1.1. Procés Maester (Pare)

Actua com el nucli central i gestor de recursos. Per garantir la reactivitat del sistema, el Maester s'ha dissenyat internament amb multifil (*multithreading*):

- **Fil Principal (*Main Thread*):**
 - **Responsabilitat:** Gestiona la Interfície d'Usuari i la lògica d'alt nivell.
 - **Funcionament:** Llegeix comandes de l'usuari (*readUntil*), interpreta la sintaxi i decideix si l'acció es pot resoldre localment o si requereix delegar una missió a un *Envoy*. També coordina la neteja de recursos al tancament.
- **Fil de Xarxa (*hiloLoop*):**
 - **Responsabilitat:** Actua com a Servidor TCP.
 - **Funcionament:** Executa un bucle infinit amb la crida *accept()*. Com que aquesta crida és bloquejant, és imprescindible aïllar-la en un fil propi per no congelar el terminal de l'usuari. Quan rep una connexió, llegeix la trama i la processa (sigui per a consum propi o per fer de *hop*).
- **Fil de Temporitzador (*hiloTimeout*):**
 - **Responsabilitat:** Garantir la integritat temporal del protocol.
 - **Funcionament:** Es desperta cada segon (*sleep(1)*) per revisar dues llistes: les peticions d'aliança rebudes pendents de resposta i els *Envoys* que han sortit de missió. Si superen els 120 segons, executa les accions de neteja i timeout.

1.1.2. Consell d'Emissaris (Fills - *Envoys*)

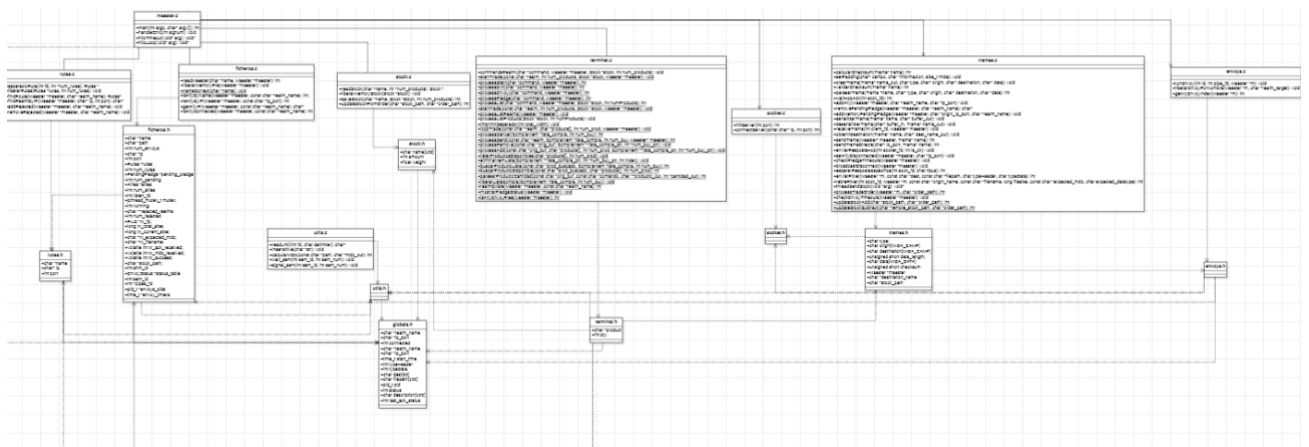
Els *Envoys* són processos independents creats mitjançant *fork()* a l'inici de l'execució.

- **Responsabilitat:** Actuen com a Clients TCP. S'encarreguen de la part "pesada" i bloquejant de les comunicacions sortints (connectar-se a un altre *socket*, fer el handshake, enviar fitxers binaris).
- **Cicle de Vida:** Es mantenen en un bucle infinit bloquejats en un *read()* de la seva *pipe*, esperant ordres. Això minimitza el consum de CPU quan no treballen.

1.1.3. Esquema de comunicació (IPC)

Per comunicar el Maester amb els seus *Envoys* s'ha optat per un model híbrid:

- **Downstream** (Maester → *Envoy*): S'utilitzen *pipes* sense nom. És el mecanisme més eficient per enviar instruccions estructurades (*Mission*).
- **Upstream** (*Envoy* → Maester): S'utilitza Memòria Compartida (SHM). Això permet al Maester consultar l'estat de tots els *Envoys* instantàniament (*envoy status*) sense haver d'esperar a llegir de múltiples pipes o gestionar senyals asíncrons complexos.



1.2. Estructures de dades

Per suportar la concurrència i la comunicació IPC, s'han dissenyat estructures específiques que garanteixen l'alineació de memòria i la facilitat de transmissió.

1.2.1. Struct Maester (Gestor global)

És l'estructura que manté l'estat global del node.

```
typedef struct Maester{
    char *name;
    char *path;
    int num_envoys;
    char *ip;
    int port;

    Rutas *rutas;
    int num_rutas;

    PendingPledge *pending_pledge;
    int num_pending;
    Allies *allies;
    int num_allies;

    int listen_fd;

    char **rejected_realms; // Array de strings amb els noms
    int num_rejected;

    pthread_mutex_t mutex;
    int running;

    int rx_fd; // FD del fitxer
    long rx_total_size; // Mida total esperada
    long rx_current_size; // Bytes rebuts fins ara
    char *rx_expected_md5; // MD5 que esperem
    char *rx_filename; // Nom del fitxer

    // Per a l'EMISSOR (Sincronització amb fils)
    volatile int tx_ack_received; // 1 = Hem rebut ACK 0x31
    volatile int tx_md5_received; // 1 = Hem rebut ACK 0x32
    volatile int tx_success; // 1 = OK, 0 = KO

    char *stock_path;

    int shm_id; // ID Memòria Compartida
    EnvoyStatus *status_table; // Puntero a la tabla en SHM
    int sem_id; // ID Semáforos para exclusión mutua
    int *pipes_fd; // Descriptores de escritura para Pipes
    pid_t *envoys_pids; // PIDs de los hijos para limpieza final
    time_t *envoy_timers; // Array per guardar el timestamp d'inici de missió
}Maester;
```

Centralitzar tots els descriptors de fitxers i *ids* de memòria en una única estructura permet passar-la fàcilment com a argument als diferents fils (*pthread_create*) i funcions, evitant l'ús excessiu de variables globals.

1.2.2. Struct Mission (Protocol de *pipe*)

Aquesta estructura defineix el protocol de comunicació interna pare-fill.

```
typedef struct {
    int typeHeader;
    int typeData;
    char dest[20];
    char filepath[100];
} Mission;
```

En lloc d'enviar *bytes* solts per la *pipe*, enviem una estructura de mida fixa (*sizeof(Mission)*). Això garanteix lectures a la *pipe* i evita problemes de sincronització o lectura parcial de comandes.

1.2.3. Struct EnvoyStatus (Estat compartit)

Estructura allotjada a la Memòria Compartida.

```
typedef struct {  
    pid_t pid;  
    int status;  
    char description[100];  
    int last_ack_status;  
} EnvoyStatus;
```

Permet desacoblar la lectura de l'estat de l'execució. El Maester pot mostrar l'estat dels *Envoys* a l'usuari en temps real sense necessitat d'interrompre els processos fills ni esperar que aquests enviïn un missatge d'actualització.

1.2.4. Struct Rutas (Gestor routes)

És una de les estructures que prové de l'estructura global Maester.

```
typedef struct Rutas {  
    char *name;  
    char *ip;  
    int port;  
} Rutas;
```

Permet guardar cada ruta que el regne té. Tenint en compte la informació aportada pel fitxer *.dat*.

1.2.5. Struct Stock (Gestor stock)

És la estructura de gestió general de *stock*.

```
typedef struct {  
    char name[100];  
    int amount;  
    float weight;  
} Stock;
```

Permet guardar els recursos (*stock*) que cada regne té per poder controlar els productes obtinguts.

1.2.6. Struct ComprarResult(gestor temporal stock)

És una estructura de pas.

```
typedef struct {  
    char *product;  
    int qty;  
} ComprarResult;
```

Permet guardar temporalment el producte amb el qual és vol fer un *start trade*.

1.2.7. Struct Trama(gestor de trames)

És una estructura on definim el protocol de comunicació que utilitzarem (una trama amb 6 camps de mida fixe).

```
typedef struct {  
    char type; // 1 Byte  
    char origin[MIDA_CAMP]; // 20 Bytes  
    char destination[MIDA_CAMP]; // 20 Bytes  
    unsigned short data_length; // 2 Bytes  
    char data[MIDA_DATA]; // 275 Bytes  
    unsigned short checksum; // 2 Bytes  
} Trama;
```

Permet controlar els camps i mides necessàries de les trames, tan per enviar com rebre.

1.2.8. Struct StockArgs(gestor temporal enviament stock)

És una estructura temporal per *trade*.

```
typedef struct {  
    Maester *maester;  
    char *destination_name;  
    char *stock_path;  
} StockArgs;
```

Permet tenir controlats a les trames quan *stock* es vol intercanviar entre regnes.

1.2.9. Struct Allies (gestor d'aliances)

És una estructura per controlar les aliances.

```
typedef struct {
    char *realm_name;
    char *ip_port;
    int connected; // Per saber si l'aliat està actiu o desconectat
} Allies;
```

Permet guardar les aliances obtingudes i si aquests regnes segueixen actius o no.

1.2.10. Struct PendingPledge (gestor temporal de *pledge*)

És una estructura per controlar les peticions de *pledge* al regne.

```
typedef struct {
    char *realm_name;
    char *ip_port;
    time_t start_time; // Guardem el temps d'enviament per controlar el timeout
} PendingPledge;
```

Permet conèixer l'estat d'una petició i controlar el *timeout*.

1.3. Recursos del sistema utilitzats

La selecció de recursos s'ha fet prioritzant la robustesa i l'eficiència en un entorn Linux POSIX.

1.3.1. Gestió de processos i fils

- **fork() i exec():** Utilitzats per crear els processos *Envoy* i per executar la comanda externa *md5sum*. L'ús de processos separats per als *Envoys* proporciona aïllament de memòria: si un *Envoy* pateix un error crític (*crash*), no tomba el servidor principal.
- **Threads (pthread):** Utilitzats per a les tasques de manteniment del Maester (*Timer* i *Accept*). S'han escollit fils en lloc de processos per aquestes tasques perquè necessiten compartir l'espai de memòria del Maester (accés a la llista d'aliats, rutes i configuració) de manera nativa i ràpida.

1.3.2. Comunicació entre processos (IPC)

- **Pipes:** S'ha triat aquest mecanisme per a la comunicació Maester → Envoy per la seva simplicitat i fiabilitat en relacions pare-fill. En ser unidireccionals i preservar l'ordre de les dades (FIFO), són ideals per a una cua de tasques.

- **Shared Memory** : Utilitzada per compartir l'*array EnvoyStatus*. És el mecanisme d'IPC més ràpid disponible, ja que evita còpies de dades entre l'espai de nucli i usuari un cop mapejada.

1.3.3. Sincronització i exclusió mútua

Donada la naturalesa concurrent del sistema, s'han implementat mecanismes estrictes per evitar Race Conditions:

- **Semàfors** : S'ha creat un conjunt de semàfors (*semget*) per gestionar recursos crítics inter-procés.
 - Semàfor 0 (SHM Access): Protegeix l'escriptura a la taula d'estats. Evita que el Maester llegeixi un estat "corrupte" mentre un *Envoy* l'està actualitzant.
 - Semàfor 1 (Database Access): Protegeix l'accés al fitxer *stock.db*. Això és vital si, per exemple, un *Envoy* està venent un producte (restant estoc) mentre el Maester està llistant els productes per pantalla (llegint estoc).

S'ha utilitzat la *flag SEM_UNDO*. Això garanteix que si un procés mor inesperadament mentre té el semàfor agafat, el Sistema Operatiu l'alliberarà automàticament, evitant *deadlocks* eterns.

- **Mutex (*pthread_mutex*)**: Utilitzat dins del procés Maester per protegir les llistes enllaçades globals (*pending_pledges*) quan hi accedeixen simultàniament el fil principal i el fil del temporitzador.

1.3.4. Xarxa i sockets

- **Sockets TCP/IP (AF_INET, SOCK_STREAM)**: S'ha escollit TCP per garantir l'entrega ordenada i sense errors dels paquets, requisit indispensable per a la transferència de fitxers binaris (imatges i bases de dades).
- **Gestió d'Errors de Xarxa**: S'ha implementat lògica per detectar desconnexions (*read* retorna 0) i errors de connexió, actualitzant l'estat dels *Envoys* a FREE immediatament per no deixar recursos bloquejats.

1.4. Implementacions Opcionals i Millores de Robustesa

Més enllà dels requisits bàsics, s'han implementat característiques avançades per garantir que el sistema sigui més robust:

1.4.1. Clean Shutdown (Tancament Net)

S'ha dissenyat un protocol de tancament específic al main per evitar els habituals *Segmentation Faults* en aturar sistemes multiprocés. L'ordre estricte implementat és:

- Cancel·lar i esperar els fils (*pthread_join*).
- Enviar *SIGTERM* als *Envoys* i esperar la seva mort (*waitpid*).
- Alliberar recursos IPC (SHM i Semàfors).
- Alliberar memòria dinàmica (*Heap*).

Això garanteix que cap fil intenti accedir a memòria que ja ha estat alliberada.

1.4.2. Gestió Avançada de Memòria (Valgrind Clean)

S'ha realitzat un esforç de depuració exhaustiu per assegurar 0 bytes de fuga de memòria (*memory leaks*). S'ha utilitzat *memset* per inicialitzar estructures abans d'enviar-les per pipes (evitant enviar brossa de memòria) i s'han alliberat meticulosament tots els buffers de recepció de fitxers.

1.4.3. Actualització d'Estoc en Temps Real (Consistency)

S'ha afegit la funcionalitat perquè, en realitzar una compra (*start trade*), el sistema actualitzi automàticament la còpia local del catàleg del venedor. Això assegura que si l'usuari torna a fer *start trade* immediatament, veurà la quantitat d'estoc real (disminuïda) sense necessitat de tornar a descarregar el catàleg remot.

2. Problemes Observats

Durant el desenvolupament de la Fase 4, la introducció de processos fills (*Envoys*) i la memòria compartida va afegir una capa de complexitat que va fer aparèixer nous problemes de sincronització i gestió de memòria. A continuació es detallen els més significatius i com s'han resolt.

2.1. Segmentation Fault al tancament

Quan s'executava la comanda *CTRL+C* per tancar el Maester, el programa sovint finalitzava amb un error de *Segmentation Fault (SIGSEGV)*. Això passava de manera aleatòria, però freqüent.

Es tractava d'una condició de carrera (*Race Condition*) durant l'alliberament de recursos. El fil principal (*main*) eliminava la Memòria Compartida (*shmctl RMID*) i els semàfors, però el fil del temporitzador (*hiloTimeout*) encara estava actiu. Si el temporitzador intentava llegir l'estat d'un *Envoy* (*status_table*) just després que el *main* hagués esborrat la memòria, accedia a una adreça invàlida.

Per solucionar-ho, es va reestructurar completament la rutina de tancament (*shutdown*) al *main* per seguir un ordre estricte de dependències:

1. Aturar els fils secundaris (*pthread_cancel* i *pthread_join*).
2. Aturar els processos fills (*kill SIGTERM* i *waitpid*).
3. Un cop no hi ha ningú actiu, alliberar la SHM i els Semàfors.
4. Alliberar la memòria dinàmica (*Heap*).

2.2. Bloqueig d'envoys per manca de resposta

Si un *Envoy* enviava una petició (*pledge* o *trade*) i el servidor destí es desconnectava, queia o simplement no responia, l'*Envoy* local es quedava en estat *ON_MISSION* indefinidament, esperant una lectura al *socket* que mai arribava. El Maester només controlava el temps de les peticions rebudes, no de les enviades.

Es va solucionar implementant un sistema de *Timeout Local*. Es va afegir un *array time_t *envoy_timers* a l'estructura del Maester. Quan un *Envoy* surt, es guarda la marca de temps. La funció *checkEnvoyTimeouts* revisa cada segon si algun *Envoy* porta més de 120 segons fora. Si és així, es marca com a *FREE* a la taula d'estats i es notifica a l'usuari, permetent reutilitzar l'*Envoy* per a noves tasques.

2.3. Leaks de memòria i dades no inicialitzades

L'eina Valgrind reportava dos tipus d'errors:

- *Syscall param write points to uninitialised byte(s)*: En enviar estructures *Mission* per la *pipe*.
- *Definetly lost*: Memòria perduda en rebre fitxers.

Les estructures en C tenen "padding" (espais buits per alineació de memòria) que contenen brossa si no s'inicialitzaven.

Les variables que guardaven el nom del fitxer rebut (*rx_filename*) i el seu MD5 s'assignaven amb *strdup* però no s'alliberaven al final de l'execució.

Això s'ha solucionat mitjançant l'ús de *memset(&mission, 0, sizeof(Mission))* abans d'escriure a la pipe per garantir que tota l'estructura està neta; i afegint els *free()* corresponents a les variables de recepció de fitxers dins la funció *liberarMemoryFile*. Gràcies a aquest dos canvis el resultat final del *valgrind* va ser 0 bytes *lost* i 0 errors.

2.4. Falsos Timeouts instantanis

En implementar el control de temps dels *Envoys*, saltava el missatge de "TIMEOUT" immediatament després d'enviar la missió.

Es va arribar a la conclusió de que era degut a que l'*array* de timers s'inicialitzava a 0. La funció *difftime(ara, 0)* calculava la diferència des de l'any 1970, donant un valor gegant que superava els 120 segons. A més, hi havia una petita finestra de temps on l'*Envoy* estava ON_MISSION però el *timer* encara no s'havia actualitzat.

Es va afegir una guarda de seguretat per solucionar el problema: si el valor del *timer* és 0, la funció de comprovació ignora l'*Envoy*. També ens vam assegurar de resetejar el *timer* a 0 sempre que un *Envoy* torna a l'estat FREE (ja sigui per èxit o per error), garantint la consistència en futures missions.

3. Estimació temporal

El desenvolupament integral del projecte "The Citadel System" s'ha dut a terme durant tot el semestre acadèmic, començant el 13 d'octubre de 2025 i finalitzant amb l'entrega de la Fase 4 l'1 de febrer de 2026.

Aquesta estimació recull el temps total dedicat per cadascun dels integrants de l'equip, acumulant la feina realitzada a les Fases 1, 2, 3 i afegint-hi la complexitat de l'arquitectura distribuïda i concurrent de la Fase 4.

3.1. Investigació

3.1.1. Fase 1 (5h)

Temps dedicat a comprendre les crides al sistema bàsiques (*read*, *write*, *open*) i la gestió de memòria dinàmica.

3.1.2. Fase 2 (6h)

Estudi de la llibreria de Sockets (*sys/socket.h*), estructures de connexió (*sockaddr_in*) i el protocol TCP/IP bàsic.

3.1.3. Fase 3 (10h)

Aprenentatge sobre l'ús de *fork*, *exec* per a eines externes (*md5sum*) i introducció als fils (*pthread*s) per a la gestió de *stock*.

3.1.4. Fase 4 (8h)

Aprofundiment en els mecanismes IPC de System V. Estudi detallat de la Memòria Compartida (*shmget*, *shmat*) i la sincronització amb Semàfors (*semop* amb flag *SEM_UNDO*). També inclou l'aprenentatge avançat de depuració amb *Valgrind*.

Temps total estimat (per estudiant): 29 hores

3.2. Disseny

3.2.1. Fase 1 (4h)

Planificació de l'arquitectura modular inicial i disseny d'estructures bàsiques (Maester, Stock).

3.2.2. Fase 2 (5h)

Disseny del protocol de comunicació (estructura Trama de mida fixa) i la màquina d'estats per a la connexió.

3.2.3. Fase 3 (7h)

Disseny de la lògica d'intercanvi de fitxers i el sistema d'encaminament (*routing*).

3.2.4. Fase 4 (5h)

Reenginyeria del sistema per passar d'un model seqüencial a un de concurrent. Disseny de l'arquitectura Pare-Fill (*Envoys*), el protocol de comunicació interna via pipes (*struct Mission*) i la taula d'estats compartida.

Temps total estimat (per estudiant): 21 hores

3.3. Implementació

3.3.1. Fase 1 (12h)

Codificació del parser de configuració, bucle del terminal i comandes locals.

3.3.2. Fase 2 (15h)

Implementació del servidor de *sockets*, l'*handshake* inicial i l'enviament de trames bàsiques.

3.3.3. Fase 3 (22h)

Desenvolupament de la transferència de fitxers binaris, càlcul de MD5 i lògica de comerç (*trade*).

3.3.4. Fase 4 (20h)

Implementació del mòdul *d'Envoys* (processos fills), lògica de sincronització amb semàfors, detecció de *timeouts* distribuïts i protocol de tancament segur (*Clean Shutdown*).

Temps total estimat (per estudiant): 69 hores

3.4. Testing

3.4.1. Fase 1 (3h)

Proves de lectura de fitxers i validació de comandes locals.

3.4.2. Fase 2 (6h)

Proves de connectivitat bàsica entre dos terminals (Client-Servidor).

3.4.3. Fase 3 (10h)

Proves d'intercanvi de fitxers i simulació d'errors de transmissió.

3.4.4. Fase 4 (15h)

Aquesta ha estat la fase més intensiva de proves degut a la concurrència. Inclou:

Validació de l'escenari global de 5 nodes.

Depuració exhaustiva amb *Valgrind* fins a aconseguir 0 *memory leaks* i 0 errors.

Proves d'estrès de xarxa i gestió de senyals (*CTRL+C*) per evitar processos zombis o *Segmentation Faults*.

Temps total estimat (per estudiant): 34 hores

3.5. Documentació

Fase 1 (2h)

Redacció de la primera memòria.

Fase 2 (3h)

Documentació del protocol de trames i evolució del projecte.

Fase 3 (6h)

Redacció de la memòria de la Fase 3 i actualització de diagrames.

Fase 4 (6h)

Redacció de la memòria final de projecte, justificació de decisions de disseny concurrent i documentació interna del codi per a l'entrega final.

Temps total estimat (per estudiant): 17 hores

TEMPS TOTAL ESTIMAT (PER ESTUDIANT): 170 hores

4. Conclusions i propostes de millora

4.1. Conclusions

La finalització de la Fase 4 suposa la culminació del projecte "The Citadel System", havent assolit amb èxit l'objectiu principal: desenvolupar una arquitectura distribuïda, concurrent i robusta capaç de simular les comunicacions complexes entre els regnes.

Les principals conclusions extretes d'aquesta última etapa són:

4.1.1. Canvi de Paradigma Arquitectònic

El pas més significatiu ha estat la transició d'un model seqüencial a un model multiprocés. La implementació dels *Envoys* ha permès desacoblar la lògica de negoci de les operacions de xarxa bloquejants. Això ha demostrat la importància de la concurrència en sistemes operatius per garantir que el node principal (Maester) es mantingui sempre receptiu a noves connexions o comandes de l'usuari.

4.1.2. Domini de la Comunicació Inter-Procés (IPC)

S'ha aprofundit en l'ús de mecanismes de baix nivell com la Memòria Compartida i els Semàfors. Hem après que l'eficiència de la memòria compartida ve acompanyada de la responsabilitat de gestionar l'exclusió mútua estrictament; un error de sincronització pot corrompre tot l'estat del sistema.

4.1.3. Robustesa i Estabilitat

Un dels grans aprenentatges ha estat la gestió d'errors i senyals. La implementació del *Clean Shutdown* (capturant *SIGINT*) i la detecció de *timeouts* distribuïts ha transformat un prototip acadèmic en un sistema estable que no deixa processos zombis ni recursos bloquejats, fins i tot en situacions de fallada de xarxa.

4.1.4. Qualitat del Codi

L'ús intensiu d'eines de depuració com *Valgrind* ens ha permès entregar un programari lliure de fuites de memòria (0 *leaks*). Això posa de manifest la importància de gestionar manualment el cicle de vida de la memòria dinàmica en C.

En resum, el projecte ens ha proporcionat una visió pràctica i profunda de com els sistemes operatius gestionen els processos, la memòria i l'entrada/sortida, consolidant els conceptes teòrics de l'assignatura.

4.2. Propostes de millora

Tot i que el sistema actual és funcional i compleix els requisits, identifiquem diverses àrees on es podria continuar treballant per apropar-lo a un entorn de producció real:

4.2.1. Encaminament Dinàmic

Actualment, les rutes són estàtiques definides als fitxers .dat. Una millora significativa seria implementar un protocol d'encaminament dinàmic (similar a *RIP* o *OSPF* simplificat) on els Maesters intercanviïn periòdicament la seva taula de veïns. Això permetria al sistema adaptar-se automàticament si un node cau, buscant camins alternatius sense reiniciar la configuració.

4.2.2. Seguretat i Encriptació

El protocol actual envia les dades (incloent-hi imatges i estoc) en text clar o binari sense xifrar. Implementar una capa de seguretat *SSL/TLS* (o un xifratge simètric bàsic tipus *XOR/AES*) seria fonamental en un context "bèl·lic" com el de Joc de Trons per evitar que els missatgers siguin interceptats.

4.2.3. Recàrrega de Configuració en Calent

Ara mateix, per canviar una ruta o afegir un aliat, cal reiniciar el Maester. Utilitzant el senyal *SIGHUP*, podríem fer que el Maester rellegís el fitxer de configuració sense aturar el servei ni tallar les connexions actives.

4.2.4. Interfície Gràfica de Monitorització

Donat que ja disposem d'una taula d'estat a la Memòria Compartida, seria relativament senzill crear un procés extern (visor) que, connectant-se a la mateixa SHM, mostrés en temps real un mapa gràfic o un panell de control (*dashboard*) amb l'estat de tots els *Envoys* i les connexions actives, millorant l'experiència d'usuari.

5. Tema de la pràctica

Aquesta pràctica està inspirada per la sèrie de televisió *Joc de Trons* (*Game Of Thrones*), que està ubicada en un món fictici amb semblances al segle XV. Les diferents cases (jerarquies familiars) viuen majoritàriament al continent de Ponent (*Westeros*), que recorda a la geografia de Gran Bretanya, i l'objectiu és obtenir el màxim poder mitjançant territoris, aliances o or per asseure's al Tron de Ferro. La majoria d'aquests objectius s'aconsegueixen a través de batalles i pactes on es poden intercanviar exèrcits, aliments, vaixells, armes... O potser ous de dracs.

Les diferents cases que podem veure a la sèrie que són mencionades a la nostra pràctica es troben *Stark* (*Winterfell*), *Baratheon* (*KingsLanding/RedKeep*), *Arryn* (*TheVale/Eyrie*), *Targaryen* (*Dragonstone*) i *Velaryon* (*Driftmark*).

Altres figures que podem veure són els *Envoys*, que podríem anomenar els "diplomàtics de cada regne", on moltes vegades són enviats en comptes del cap del regne per pactar o declarar guerres. En la nostra pràctica, aquests són els processos fills encarregats de viatjar (connectar-se) als altres regnes.

Transaccions que es van veure implementades que poden recordar a la sèrie, serien els "*hops*", que com a tal podríem definir-ho com la intermediació d'un aliat (o més), de cada costat per aconseguir que dos regnes puguin crear una aliança per efectuar intercanvis com de productes, que més endavant podria ser una aliança forta per lluitar contra altres regnes. No només això, sinó que l'enviament de segells (sigils) podríem veure l'analogia amb *Joc de Trons* quan els diferents regnes porten la bandera amb el segell reial tant quan surten del territori com a les guerres.

A continuació, es detalla la relació dels noms triats per als processos del sistema amb la temàtica de la sèrie:

- **Maester (El Mestre):** En l'univers de *Joc de Trons*, els Maesters són els savis, consellers i metges assignats a cada castell. Són els encarregats de la comunicació (envien els corbs missatgers), guarden el coneixement i aconsellen al senyor del castell. A la pràctica, aquest és el nom del procés principal (pare). Té tot el sentit, ja que és qui "sap" ho gestiona tot: té el mapa de les rutes, coneix qui són els aliats, gestiona l'inventari del castell i dona les ordres als missatgers.

- *Envoy* (L'Emissari): A la sèrie, quan un rei vol enviar un missatge important, una oferta de pau o una declaració de guerra, envia un emissari o ambaixador. Aquests personatges viatgen físicament fins al castell enemic o aliat per entregar el missatge en nom del seu senyor. A la pràctica, aquests són els processos fills. Són creats pel Maester per "sortir" del node (connectar-se via *socket*) i portar la petició (*Mission*) a un altre lloc, tornant després amb la resposta.
- *The Citadel* (La Ciutadella): És el nom del sistema global. A la sèrie, la Ciutadella és la seu central de l'ordre dels Maesters, el lloc on s'emmagatzema tot el coneixement del món i des d'on s'envien els corbs blancs per anunciar el canvi d'estacions (com l'hivern). Simbolitza la xarxa que connecta tots els regnes a través de la informació.

6. Bibliografia

La Salle Campus Barcelona. (2025). *Apunts de Sistemes Operatius: Programació de Sistemes UNIX*. Departament d'Enginyeria de Sistemes Informàtics. La Salle - Universitat Ramon Llull.)Consultat durant tot el desenvolupament).

- Referència principal per a l'ús de crides al sistema (*fork*, *exec*, *pipe*) i gestió de fitxers.

Valgrind Developers. (2024). *Valgrind User Manual: Memcheck*. Recuperat de <https://valgrind.org/docs/manual/mc-manual.html>

- Utilitzat per a la detecció i correcció de fuites de memòria i errors d'inicialització a la Fase 4.

Rivest, R. (1992). The MD5 Message-Digest Algorithm (RFC 1321). Internet Engineering Task Force (IETF). Recuperat de <https://datatracker.ietf.org/doc/html/rfc1321>

- Base teòrica per a la comprensió i ús de l'eina md5sum per a la validació d'integritat de fitxers.

Google. (2026). *Gemini* (Versió gener 2026) [Model de llenguatge gran]. Recuperat de <https://gemini.google.com/>

- Utilitzat com a assistent de suport per a la depuració de codi (Valgrind), optimització de l'arquitectura multiprocés i revisió de la documentació tècnica.